Step 7 — Create PR & submit

Submit a PR from `golden-solution` into `main` with exactly three commits: `[sol]`, `[f2p]`, and `[meta]`. **Title:** `[GOLDEN SOLUTION]` + the original PR title.

PR body

Your PR description should clearly explain three things: the problem[1], your approach[2], and your testing strategy[3]. A well-documented PR makes it much easier to write a high-quality LLM prompt.

1. What issue does the original PR address? What was broken or missing?
2. What did you change and why? How does your golden solution differ from the original?
3. What tests did you write or modify? What behaviors do they verify?

Think of the PR as your notes

This description should be able to validate the `problem_statement` in `metadata.json`. The clearer you write it, the more confident you will be about your problem statement.

Validate with Helix Pre-Validation

When you open the PR, the **Helix Pre-Validation** GitHub Action runs automatically. You do not need to validate inside Docker manually — just confirm the workflow passes.

1

Open the PR

Open the PR from `golden-solution` into `main`.

2

Check the Checks tab

Navigate to the **Checks** tab on the PR page.

3

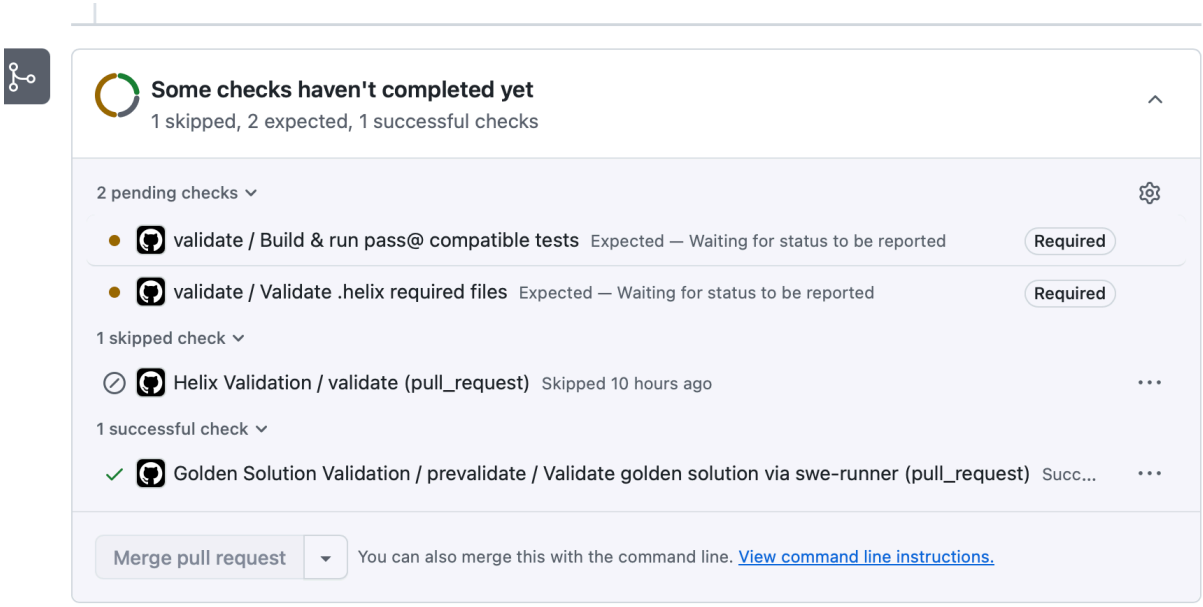
Find the workflow

Find the **Helix Pre-Validation** workflow run.

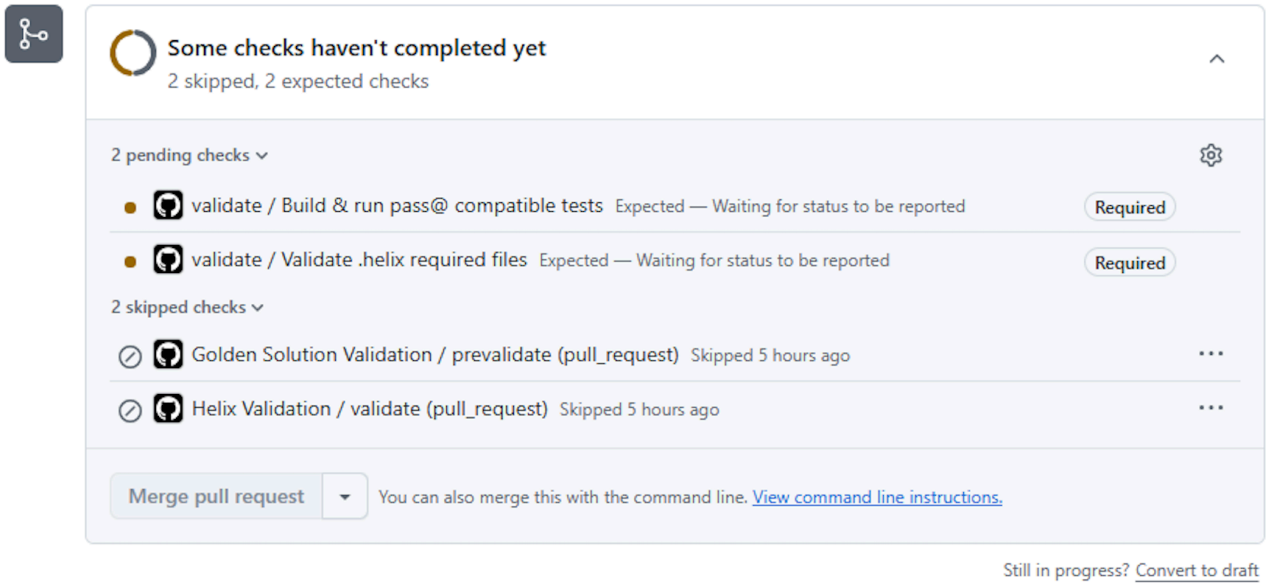
4

Confirm it passes

Confirm it shows a green checkmark.



Golden Solution Validation passed — the PR is ready to merge.



Checks skipped or stalled — do not merge. Investigate why checks are not running.

No local Docker needed

The Helix Pre-Validation workflow handles all validation for you. If it passes, you're good to go.

Common failure modes

If the workflow fails, click into the execution logs to identify the error. The validation system uses three commits: **base commit** (the most recent commit on the default

branch), `[sol]` (your golden solution), and `[f2p]` (your fail-to-pass tests).

Here are the most common errors and how to fix them:

GIT_RESET_FAILED — Commit could not be checked out

GIT_APPLY_FAILED — Patch could not be applied

P2P_TESTS_FAILED — Pass-to-pass tests failed on base commit

F2P_TESTS_SHOULD_FAIL — Fail-to-pass tests passed when they shouldn't have

GOLDEN_VALIDATION_FAILED — Tests failed with the solution applied

Commit tips

```
git update-index --chmod=+x .helix/run-tests-eval.sh
git add .helix
git commit -m "your message"
```

Copy

When working on the `docker-golden-solution` branch (during Environment Setup), always use `git update-index` to set the executable bit on `run-tests-eval.sh` before committing.

Submit on HAI

On the **HAI platform**, submit a screenshot of your validation checks passing.

Pre-submit scorecard — map to the reviewer rubric

Before you submit, walk down the table. If you can't write a sentence next to each box explaining why the reviewer can't tick it, that box is your risk.

Reviewer checkbox	Your self-check that prevents it
Did not pass GitHub checks	Helix Pre-Validation green on the `golden-solution` PR.
There are missing tests	Every `problem_statement` sentence maps to ≥1 F2P test (see Step 5 scope rubric).

Reviewer checkbox	Your self-check that prevents it
Fail-to-pass tests are overfitting	Every F2P assertion derives from a prompt sentence — no hidden requirements (see Step 5 "Don't go beyond").
Trivial fail-to-pass test cases	Tests assert behavior, not symbol existence (see Step 5 "Tests must verify behavior").
Trivial pass-to-pass tests	P2P set comes from existing tests on `main`, not new additions.
Problem statement is not related to PR	5/5 on the `problem_statement` rubric — self-contained, no leakage.
Problem statement is too vague	5/5 — names new symbols where needed, contract not recipe.
Solution code is incomplete	Workflow 3 contract check passes — F2P + P2P all pass on base + `[f2p]` + `[sol]`.
Solution code is incorrect	Same — run F2P locally against `[sol]` before submitting.
Task not suitable for Helix (too complex)	Pre-claim validity scorecard scored 5/5 — real-logic SLOC ≤ 500.
Task not suitable for Helix (too simple)	Same — real-logic SLOC ≥ 25.
No issues — perfect	All of the above clean.

Submission floor

Do not submit if any row is unchecked. A row you can't defend predicts a sendback.

Previous Next